

Optimisations

Génération, Compila-
tion et Analyse Statique

Hugues Cassé <casse@irit.fr>

2022





Introduction

- optimisations non nécessaires $\implies$ code exécutable
- différents critères :
 - temps d'exécution (plus fréquent)
 - place mémoire
(environnement contraint – système embarqué)
 - déterminisme du code (temps-réel)
 - etc

Note : nom des optimisations en anglais $\implies$ correspondance avec les options des compilateurs



Outline

- ① Optimisations locales
- ② Optimisations globales
- ③ Interprétation abstraite
- ④ Conclusion



Simplifications algébriques

A $x + 0 = 0 + x = x - 0 = x$

B $0 - x = -x$

C $x \times 1 = 1 \times x = x / 1 = x$

D $x \times 0 = 0 \times x = 0 / x = 0$

E $-(-x) = x$

F $x + (-y) = x - y$

G $x \times 2^k = x \ll k$

I $x / 2^k = x \gg k$

J $x \ll 0 = x \gg 0 = x$

K $x \ll |\tau| = x \gg |\tau| = 0$

L $x^2 = x \times x$

M $x^0 = 1$

avec τ type de x , y et $|\tau|$ taille de τ en bits.



Optimisation à lucarne

sur les quadruplets

- recherche d'un modèle de simplification algébrique
- fenêtre de quelques instructions
- reconnaissance $\implies$ remplacement

sur le DAG $\implies$ insertion dans les modèles d'instructions

...		...
$v_3 \leftarrow 8$	$(v_i \leftarrow 2^k)$	$\implies v_3 \leftarrow 8$
		$v_{51} \leftarrow 3$
$v_4 \leftarrow v_1 \times v_3$	$(v_j \leftarrow v_k \times v_i)$	$v_4 \leftarrow v_1 \ll v_{51}$
...		...

NOTE : $v_3 \leftarrow 8$ sera éliminée par une autre phase d'optimisation (*dead-code elimination*)



Cas des branchements

$\text{goto } (l_i)$
 $\text{label } (l_i)$ $\implies$ $\text{label } (l_i)$

$\text{goto.}\omega \ v_i, v_j, l_k$
 $\text{goto } l_l$
 $\text{label } l_k$ $\implies$ $\text{goto.}\overline{\omega} \ v_i, v_j, l_l$
 $\text{label } l_k$

avec $\overline{\omega}$ condition inverse d' ω .

Opérations propres à une architecture

$$v_i \leftarrow v_j \times 10 \iff \begin{cases} \text{ADD } v_i, v_j, v_j \\ \text{ADD } v_i, v_i, v_i, \text{LSL } \#2 \end{cases}$$

$$v_i \leftarrow v_j \times 14 \iff \begin{cases} \text{ADD } v_i, v_j, v_j \\ \text{RSB } v_i, v_i, v_i, \text{LSL } \#3 \end{cases}$$

Limite : $\text{temps}_{\text{code optimisé}} < \text{temps}_{\text{instruction}}$

Explication :

$$10 \times x = 5 \times 2x = (1 + 4) \times 2x = 2x + 2x << 2$$

$$14 \times x = 7 \times 2x = (8 - 1) * 2x = 2x << 3 - 2x$$

Pourquoi y a-t-il de telles expressions ?

- ◆ code automatiquement généré
- ◆ utilisation intensive des macro-définitions
- ◆ conservation de la lisibilité
- ◆ résultat d'optimisations précédentes

```
#define KO (1 << 10)
#define MO (KO << 10)
```

```
...
memory_size = 4 * MO;
...
```

après expansion avec cpp :

```
...
memory_size = 4 * ((1 << 10) << 10);
...
```



Optimisation des sous-programmes

- *tail call optimization* – appel de sous-programme en fin $\implies$ branchement simple
- *inlining* – remplacement d'un appel par le code du sous-programme
 - dirigé par inline
 - taille du sous-programme $<$ seuil
- *leaf routine optimization* – simplification des sous-programme ne faisant pas d'appel
 - SP constant $\implies$ FP non utilisé
 - pas de sauvegarde du LR
- *recursion elimination* – remplacement par une boucle si possible (langages fonctionnels – OCAML)

Pipeline : ré-ordonnancement des instructions

Version 1

- (a) LDR R_1 , [FP, #4]
- (b) LDR R_2 , [FP, #16]
- (c) ADD R_3 , R_1 , R_2
- (d) LDR R_4 , [FP, #20]
- (e) SUB R_5 , R_3 , R_4
- (f) STR R_5 , [FP, # - 8]
- (g) LDR R_6 , [FP, #16]
- (h) CMP R_6 , #0
- (i) BNE I_3

Version 2

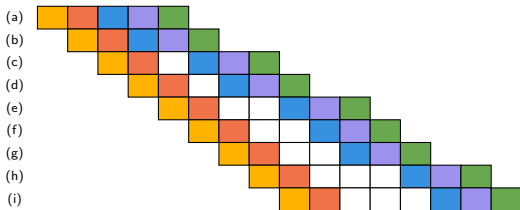
- (a) LDR R_1 , [FP, #4]
- (b) LDR R_2 , [FP, #16]
- (d) LDR R_4 , [FP, #20]
- (g) LDR R_6 , [FP, #16]
- (c) ADD R_3 , R_1 , R_2
- (h) CMP R_6 , #0
- (e) SUB R_5 , R_3 , R_4
- (f) STR R_5 , [FP, # - 8]
- (i) BNE I_3



Exécution dans le pipeline

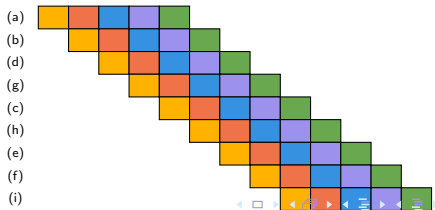
pipeline :

FE	DE	EX	ME	WB
----	----	----	----	----



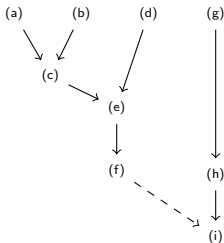
Version 1 (16 cycles)

Version 2 (13 cycles)



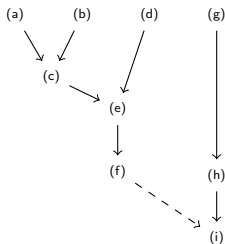


Ordonnancement par liste



- ◆ graphe de dépendance (DAG)
- ◆ sélection d'une instruction selon ses dépendances
- ◆ sélection en **largeur d'abord**
- ◆ sélection **top-down**, **bottom-up**

Table de réservation



FE	DE	EX	ME	WB
(a)				
(b)	(a)			
(d)	(b)	(a)		
(g)	(d)	(b)	(a)	
(c)	(g)	(d)	(b)	(a)
(h)	(c)	(g)	(d)	(b)
(e)	(h)	(c)	(g)	(d)
(f)	(e)	(h)	(c)	(g)
(i)	(f)	(e)	(h)	(c)
	(i)	(f)	(e)	(h)
		(i)	(f)	(e)
			(i)	(f)
				(i)

Table de réservation : modélise les conflits des instructions dans le pipeline.



Extraction de parallélisme

processeur superscalaire

- dans l'ordre
- dans le désordre (*reorder buffer*)

VLIW (*Very Long Instruction Word*)

- instructions parallèles groupées en *bundle*
- composition des *bundle* sous contrainte :
par ex. 1 mult./div., 1 accès mémoire, 1 instruction FPU, n instructions ALU

Plus il existe de parallélisme

- ⇒ efficacité de fonctionnement – ILP (*Instruction Level Parallelism*)
- ⇒ augmenter la taille des blocs

Exercice

- 1 Calculez le nombre de cycle nécessaire à l'exécution de la séquence ci-dessous.
- 2 Réordonnez ce code selon la table de réservation.
- 3 Déduisez en le nouveau temps d'exécution.

(a) SUB R3, R1, #1
(b) ADD R3, R0, R1, LSL #64
(c) LDR R3, [R2, R3, LSL #2]
(d) ADD R4, R0, #1
(e) ADD R4, R4, R1, LSL #64
(f) LDR R4, [R2, R4, LSL #2]
(g) ADD R3, R3, R4
(i) SUB R5, R0, #1
(j) ADD R5, R5, R1, LSL #64
(k) LDR R5, [R2, R5, LSL #2]
(l) ADD R4, R3, R4
(m) ADD R6, R1, #1
(n) ADD R6, R6, R1, LSL #64
(o) LDR R6, [R2, R6, LSL #2]
(p) ADD R3, R3, R6
(q) ADD R0, R3, R4



Aggrandir le BB

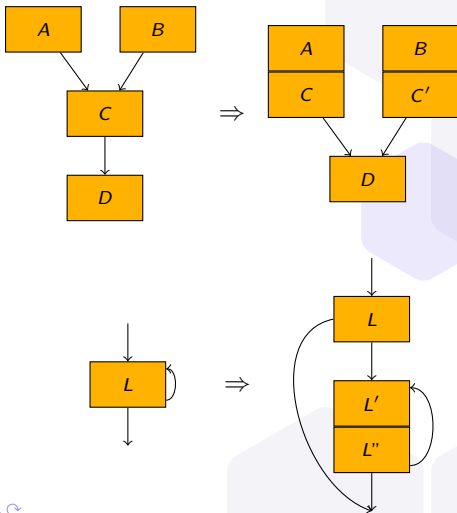
Superbloc :

- agréger des blocs proche
- bloc suivant une sélection $\Rightarrow$ dupliqué sur chaque branche

Déroutage de boucle (*loop unrolling*) :

- dupliquer une boucle N fois,
- renommer les registres selon dépendance,
- si nombre d'itération inconnu, ajouter $N - 1$ itérations.

Attention : ↗ taille du programme $\Rightarrow$
↘ efficacité du cache d'instruction





Optimiser pour les caches

- code dans boucle $\Rightarrow$ *localité spatiale* + *localité temporelle*
- données utilisées conjointement (boucle) placées dans le même block $\Rightarrow$ *localité spatiale* + *localité temporelle*
- 1 tableau $\Rightarrow$ *localité spatiale*
- code/données $\Rightarrow$ précharger un bloc avant son exécution (instruction spéciale)
 $\Rightarrow$ augmentation de la taille du code
- n fonctions utilisées conjointement
- n tableaux utilisés conjointement
 $\Rightarrow$ conflit d'utilisation du cache
 $\Rightarrow$ les faire démarrer dans des lignes différentes

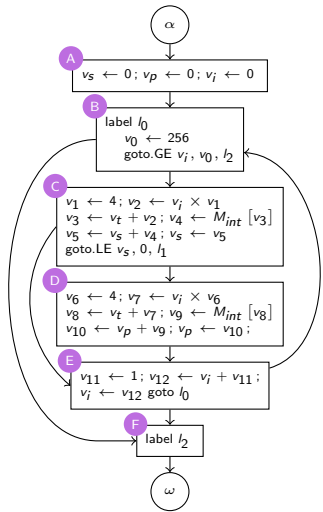
NOTE : toute optimisation réduisant la taille du code améliore le cache d'instruction



Outline

- ① Optimisations locales
- ② Optimisations globales**
- ③ Interprétation abstraite
- ④ Conclusion

Exemple : expression disponibles



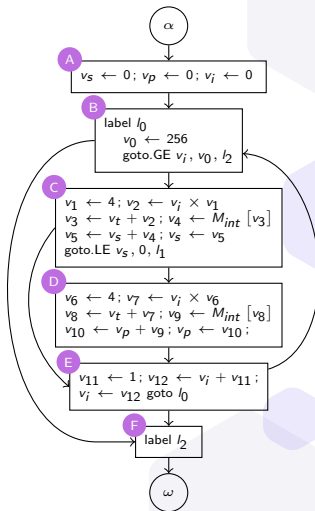
```
s = 0;
p = 0;
i = 0;
while(i < 256) {
    s += t[i];
    if(s > 0)
        p += t[i];
    i++;
}
```



Expression disponible

Optimisation :

- comme $v_1 \leftarrow 4$ (BB 3) est disponible avant BB 4
- supprimer $v_6 \leftarrow 4$
- remplacer v_6 par v_1



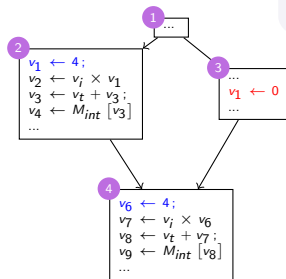
Attention !

Optimisation :

- comme $v_1 \leftarrow 4$ (BB 3) est disponible avant BB 4
- supprimer $v_6 \leftarrow 4$
- remplacer v_6 par v_1

Est-on sûr que c'est toujours possible ?

- contre-exemple
- $v_1 \leftarrow 4$?
- Doit être disponible sur tous les chemins !



Attention !

Optimisation :

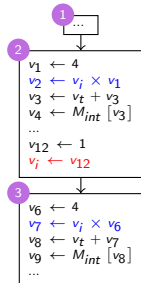
- comme $v_1 \leftarrow 4$ (BB 3) est disponible avant BB 4
- supprimer $v_6 \leftarrow 4$
- remplacer v_6 par v_1

Est-on sûr que c'est toujours possible ?

- contre-exemple
- $v_1 \leftarrow 4$?
- **Doit être disponible sur tous les chemins !**

Est-on sûr que c'est toujours possible ?

- contre-exemple
- $v_2 \leftarrow v_i \times v_1$?
- *Non car v_i est modifié !*





Analyse

Soit un CFG $G = \langle V, E, \alpha, \omega \rangle$

Pour optimiser $B \in V$,

- expressions (quadruplets) disponibles en entrée de $B - IN_B$
- $IN_B \subset \mathcal{Q}$ est un ensemble car plusieurs chemins mènent à B
- IN_B doit être complet : prendre en compte tous les chemins

Pour calculer IN_B :

- supprimer les quadruplets des v_i affectés dans B
 $v_i \leftarrow e_1, v_i \leftarrow e_2, \dots$
- supprimer les quadruplets dont un opérande est affecté dans B
 $v_j \leftarrow v_i \ \omega \ v_k, v_k \leftarrow v_l \ \omega \ v_i, \dots$
- ajouter les quadruplets de B des dernières affectations de v_i
 $v_i \leftarrow e_1$

Problèmes :

- calculer pour tous les chemins pour chaque $B \in V$!

Idée : calculer tout en parallèle!

Poussons l'analyse :

$\mathcal{D} \subseteq \mathcal{Q}$ domaines des expressions disponibles (quadruplets)

$IN_B \subseteq \mathcal{D}$ expressions en entrée de B

$OUT_B \subseteq \mathcal{D}$ expressions en sortie de B

$KILL_B \subseteq \mathcal{D}$ expressions supprimées par B

$GEN_B \subseteq \mathcal{D}$ expression ajoutées par B

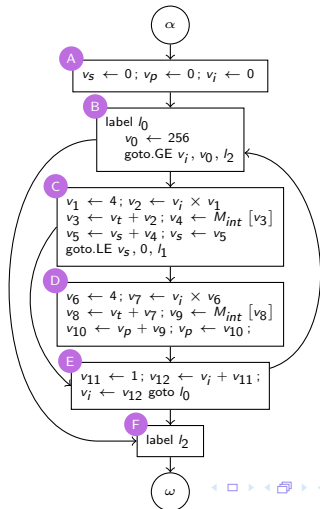
Definition

calcul des IN_B :

$$IN_B = \bigcap_{A \in PRED(B)} OUT_A$$

$$OUT_B = (IN_B \setminus KILL_B) \cup GEN(B)$$

Suite de l'exemple



$$\mathcal{D} = \{v_s \leftarrow 0, v_p \leftarrow 0, \dots\}$$

$$GEN_A = \{v_s \leftarrow 0, v_p \leftarrow 0, v_i \leftarrow 0\}$$

$$KILL_A = \{v_s \leftarrow 0, v_p \leftarrow 0, v_i \leftarrow 0, \\ v_s \leftarrow v_5, v_p \leftarrow v_{10}, v_i \leftarrow v_1, \\ v_5 \leftarrow v_s + v_4, v_{10} \leftarrow v_p + v_9, \\ v_{12} \leftarrow v_i + v_{11}\}$$

$$GEN_B = \{v_0 \leftarrow 256\}$$

$$KILL_B = \{v_0 \leftarrow 256\}$$

$$GEN_C = \{v_1 \leftarrow 4, v_2 \leftarrow v_i \times v_1,$$

$$v_3 \leftarrow v_t + v_2, v_9 \leftarrow M_{int}[v_3],$$

$$v_5 \leftarrow v_s + v_4, v_s \leftarrow v_5\}$$

$$KILL_C = \{v_1 \leftarrow 4, v_2 \leftarrow v_i \times v_1,$$

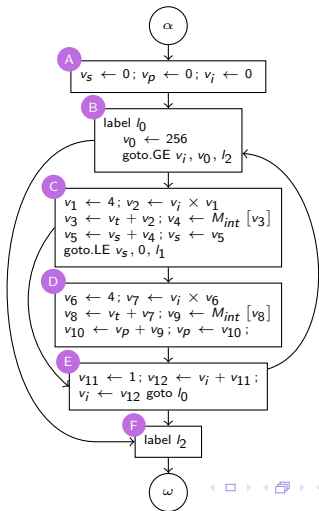
$$v_3 \leftarrow v_t + v_2, v_9 \leftarrow M_{int}[v_3],$$

$$v_5 \leftarrow v_s + v_4, v_s \leftarrow v_5,$$

$$v_s \leftarrow 0\}$$

...

Système d'équation



$$IN_\alpha = \mathcal{D}$$

$$OUT_\alpha = (IN_\alpha \setminus KILL_\alpha) \cup GEN_\alpha$$

$$IN_A = OUT_\alpha$$

$$OUT_A = (IN_A \setminus KILL_A) \cup GEN_A$$

$$IN_B = OUT_B \cap OUT_E$$

$$OUT_B = (IN_B \setminus KILL_B) \cup GEN_B$$

$$IN_C = OUT_B$$

$$OUT_C = (IN_C \setminus KILL_C) \cup GEN_C$$

$$IN_D = OUT_C$$

$$OUT_D = (IN_D \setminus KILL_D) \cup GEN_D$$

$$IN_E = OUT_D \cap OUT_C$$

$$OUT_E = (IN_E \setminus KILL_E) \cup GEN_E$$

$$IN_F = OUT_B$$

$$OUT_F = (IN_F \setminus KILL_F) \cup GEN_F$$

$$IN_\omega = OUT_F$$

$$OUT_\omega = (IN_\omega \setminus KILL_\omega) \cup GEN_\omega$$

Comment le résoudre ?

Treillis $\langle S, \sqsubseteq, \sqcup, \sqcap, \top, \perp \rangle : \forall x, y \in \mathcal{D},$

- $\perp \sqsubseteq x \sqsubseteq \top$
- $x \sqsubseteq x \sqcup y, y \sqsubseteq x \sqcup y$
- $x \sqcap y \sqsubseteq x, x \sqcap y \sqsubseteq y$

Fonction $f : S \rightarrow S$ **monotone** : $\forall x, y \in \mathcal{D}$
 $x \sqsubseteq y \implies f(x) \sqsubseteq f(y)$

Theorem

(Knaster-Tarsky) Si $\langle S, \sqsubseteq, \sqcup, \sqcap, \top, \perp \rangle$ est un **treillis** et $f : S \rightarrow S$ **monotone**, alors il existe un point fixe $X \in S$ vérifiant $f(X) = X$.

- peu importe l'ordre du calcul, f convergera vers X ,
- pour avoir le plus petit, il faut partir de $\perp$
- calculer $f^{n+1} = f(f^n)$ avec $f^0 = \perp$ jusqu'à atteindre X

Application à un seul BB

$\langle \emptyset(\mathcal{D}), \supseteq, \cap, \cup, \emptyset, \mathcal{D} \rangle$ est un treillis !

Soit $f : \mathcal{D}^{|V|} \rightarrow \mathcal{D}$ défini par :

$$\begin{aligned} f_B(OUT_X) &= OUT_B \\ &= (IN_B \setminus KILL_B) \cup GEN_N \\ &= ((\bigcap_{X \in pred(B)} OUT_X) \setminus KILL_B) \cup GEN_B \end{aligned}$$

tel que OUT_X est l'ensemble des OUT des BB de V .

Considérant que les OUT_X sont décroissants, f_B est monotone :

- $\forall X, Y, X', Y' \subseteq \mathcal{D} \wedge X \supseteq X' \wedge Y \supseteq Y'$
- $(X \cap Y) \supseteq (X' \cap Y') \rightarrow$ intersection décroissante
- $(X \cup Y) \supseteq (X' \cup Y') \rightarrow$ union décroissante
- $\setminus$ n'est généralement pas décroissante mais $KILL_B$ constant !
 $X \setminus KILL_B \supseteq X' \setminus KILL_B \rightarrow$ différence décroissante



Élargissement au CFG

Domaine élargi au CFG complet : $\mathcal{D}_G = \wp(\mathcal{D})^{|V|}$ – ensemble des OUT_B du CFG

Peut-on construire le treillis $(\mathcal{D}_G, \sqsubseteq, \sqcup, \sqcap, \top, \perp)$?

- $(X_1, X_2, \dots) \sqsubseteq (Y_1, Y_2, \dots)$ – ordre lexicographique
 $(X_1 \supset Y_1) \vee (X_1 = Y_1 \wedge ((X_2 \supset Y_2) \vee \dots))$
- $(X_1, X_2, \dots) \sqcup (Y_1, Y_2, \dots) = (X_1 \cap Y_1, X_2 \cap Y_2, \dots)$
- $(X_1, X_2, \dots) \sqcap (Y_1, Y_2, \dots) = (X_1 \cup Y_1, X_2 \cup Y_2, \dots)$
- $\top = (\emptyset, \emptyset, \dots)$
- $\perp = (\mathcal{D}, \mathcal{D}, \dots)$
- $(\mathcal{D}_G, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ est un treillis !

Élargissement à notre système d'équation

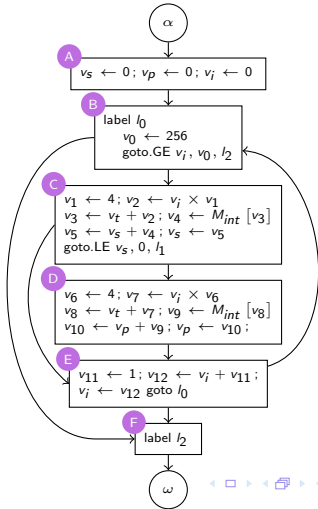
$f_G : \mathcal{D}_G \mapsto \mathcal{D}_G$ tel que

- $f_G([OUT_A, OUT_B, \dots]) = [f_A(OUT_A, OUT_B, \dots), f_B(OUT_A, \dots), \dots]$
- f_G croissant dans $\mathcal{D}_G$ car f_b (dé)croissant dans $\wp(\mathcal{D})$
- solution pour les $OUT_B =$ plus petit point fixe
 - $f_G^0 = \perp$
 - $f_G^{n+1} = f_G(f_G^n)$

Petite aide : aucune expression disponible avant α !

$$\begin{aligned} KILL_\alpha &= \mathcal{D}, \quad GEN_\alpha = \emptyset \\ \implies OUT_\alpha &= \emptyset \end{aligned}$$

Example (1)



$$IN_\alpha = \mathcal{D}$$

$$OUT_\alpha = \emptyset$$

$$IN_A = \emptyset$$

$$OUT_A = \{v_s \leftarrow 0, v_p \leftarrow 0, v_i \leftarrow 0\}$$

$$IN_B = OUT_A \cap OUT_E = OUT_A \cap \mathcal{D}$$

$$OUT_B = \{v_s \leftarrow 0, v_p \leftarrow 0, v_i \leftarrow 0, \\ v_0 \leftarrow 256\}$$

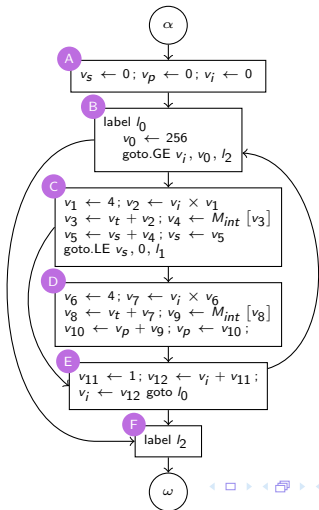
$$IN_C = OUT_B$$

$$OUT_C = \{v_p \leftarrow 0, v_i \leftarrow 0, v_0 \leftarrow 256, \\ v_1 \leftarrow 4, v_2 \leftarrow v_i \times v_1, \\ v_3 \leftarrow v_t + v_2, v_4 \leftarrow M_{int}[v_3], \\ v_5 \leftarrow v_5\}$$

$$IN_D = OUT_C$$

$$OUT_D = \{v_i \leftarrow 0, v_0 \leftarrow 256, \\ v_1 \leftarrow 4, v_2 \leftarrow v_i \times v_1, \\ v_3 \leftarrow v_t + v_2, v_4 \leftarrow M_{int}[v_3], \\ v_s \leftarrow v_5, v_6 \leftarrow 4, \\ v_7 \leftarrow v_i \times v_6, v_8 \leftarrow v_t + v_7, \\ v_9 \leftarrow M_{int}[v_8], v_p \leftarrow v_{10}\}$$

Example (2)



...

$OUT_C = \{v_p \leftarrow 0, v_i \leftarrow 0, v_0 \leftarrow 256,$
 $v_1 \leftarrow 4, v_2 \leftarrow v_i \times v_1,$
 $v_3 \leftarrow v_t + v_2, v_4 \leftarrow M_{int}[v_3],$
 $v_s \leftarrow v_5\}$

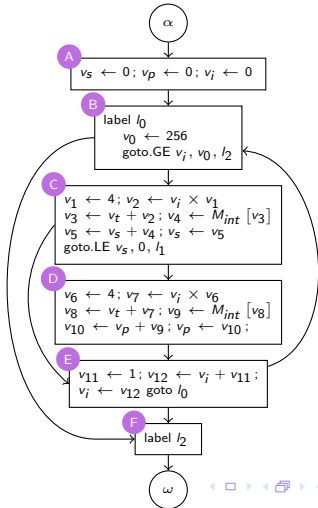
$IN_D = OUT_C$

$OUT_D = \{v_i \leftarrow 0, v_0 \leftarrow 256,$
 $v_1 \leftarrow 4, v_2 \leftarrow v_i \times v_1,$
 $v_3 \leftarrow v_t + v_2, v_4 \leftarrow M_{int}[v_3],$
 $v_s \leftarrow v_5, v_6 \leftarrow 4,$
 $v_7 \leftarrow v_i \times v_6, v_8 \leftarrow v_t + v_7,$
 $v_9 \leftarrow M_{int}[v_8], v_p \leftarrow v_{10}\}$

$IN_E = \{v_i \leftarrow 0, v_0 \leftarrow 256, v_1 \leftarrow 4,$
 $v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$
 $v_4 \leftarrow M_{int}[v_3]\}$

$OUT_E = \{v_0 \leftarrow 256, v_1 \leftarrow 4,$
 $v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$
 $v_4 \leftarrow M_{int}[v_3], v_i \leftarrow v_{12}\}$

Example (3)



...

$$OUT_A = \{v_s \leftarrow 0, v_p \leftarrow 0, v_i \leftarrow 0\}$$

...

$$OUT_E = \{v_0 \leftarrow 256, v_1 \leftarrow 4,$$

$$v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$$

$$v_4 \leftarrow M_{int}[v_3], v_i \leftarrow v_{12}\}$$

$IN_B = \emptyset$

$OUT_B = \{v_0 \leftarrow 256\}$

$$OUT_C = \{v_0 \leftarrow 256, v_1 \leftarrow 4,$$

$$v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$$

$$v_4 \leftarrow M_{int}[v_3], v_s \leftarrow v_5\}$$

$OUT_D = \{v_0 \leftarrow 256, v_1 \leftarrow 4,$

$$v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$$

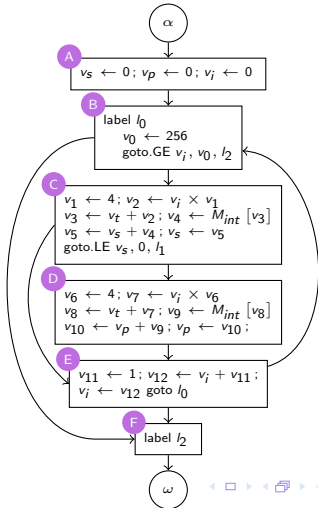
$$v_4 \leftarrow M_{int}[v_3], v_s \leftarrow v_5,$$

$$v_6 \leftarrow 4, v_7 \leftarrow v_i \times v_6,$$

$$v_8 \leftarrow v_t + v_7, v_4 \leftarrow M_{int}[v_3],$$

$$v_p \leftarrow v_{10}\}$$

Example (4)



...

$$OUT_C = \{v_0 \leftarrow 256, v_1 \leftarrow 4,$$

$$v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$$

$$v_4 \leftarrow M_{int}[v_3], v_s \leftarrow v_5\}$$

...

$$OUT_D = \{v_0 \leftarrow 256, v_1 \leftarrow 4,$$

$$v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$$

$$v_4 \leftarrow M_{int}[v_3], v_s \leftarrow v_5,$$

$$v_6 \leftarrow 4, v_7 \leftarrow v_i \times v_6,$$

$$v_8 \leftarrow v_t + v_7, v_9 \leftarrow M_{int}[v_8],$$

$$v_{10} \leftarrow v_p + v_9, v_p \leftarrow v_{10}\}$$

$$IN_E = \{v_0 \leftarrow 256, v_1 \leftarrow 4,$$

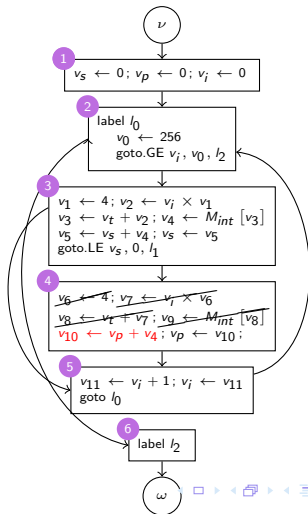
$$v_2 \leftarrow v_i \times v_1, v_3 \leftarrow v_t + v_2,$$

$$v_4 \leftarrow M_{int}[v_3], v_s \leftarrow v_5\}$$



Optimisation

expressions disponibles



$IN_A = \emptyset$

$IN_B = \emptyset$

$IN_C = \{\nu_0 \leftarrow 256\}$

$IN_D = \{\nu_0 \leftarrow 256, \nu_1 \leftarrow 4,$
 $\nu_2 \leftarrow \nu_i \times \nu_1, \nu_3 \leftarrow \nu_t + \nu_2,$
 $\nu_4 \leftarrow M_{int}[\nu_3], \nu_s \leftarrow \nu_5\}$

$IN_E = \{\nu_0 \leftarrow 256, \nu_1 \leftarrow 4,$
 $\nu_2 \leftarrow \nu_i \times \nu_1, \nu_3 \leftarrow \nu_t + \nu_2,$
 $\nu_4 \leftarrow M_{int}[\nu_3], \nu_s \leftarrow \nu_5\}$

$IN_F = \emptyset$



Algorithme

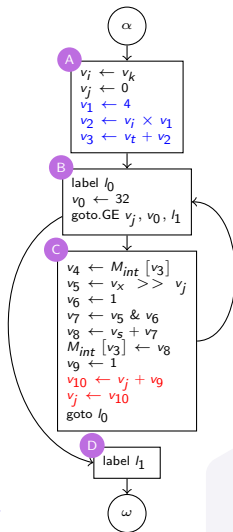
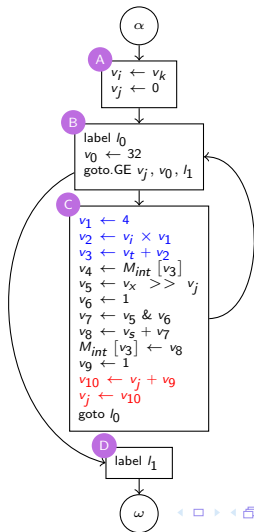
```
for  $B \in V$  do
|  $OUT_B \leftarrow \perp$ 
end
 $changed \leftarrow true$  ;
while  $changed$  do
|  $changed \leftarrow false$  ;
| for  $B \in V$  do
| |  $OUT_{old} \leftarrow OUT_B$  ;
| |  $IN \leftarrow \bigcap_{A \in PRED(B)} OUT_A$  ;
| |  $OUT_B \leftarrow (IN \setminus KILL_B) \cup GEN_B$  ;
| |  $changed \leftarrow changed \vee (OUT_{old} \neq OUT_b)$ 
| end
end
```

Algorithme avec liste de travail

```
for  $B \in V$  do
  |  $OUT_B \leftarrow \perp$ 
end
 $wl \leftarrow \{\alpha\}$ ;
while  $wl \neq \emptyset$  do
  |  $B \leftarrow x \in wl$ ;
  |  $wl \leftarrow wl \setminus \{B\}$ ;
  |  $IN \leftarrow \bigcap_{A \in PRED(B)} OUT_A$ ;
  |  $OUT_{new} \leftarrow (IN \setminus KILL_B) \cup GEN_B$ ;
  | if  $OUT_{new} \neq OUT_B$  then
    | |  $OUT_B \leftarrow OUT_{new}$ ;
    | |  $wl \leftarrow wl \cup SUCC(B)$ 
  | end
end
```



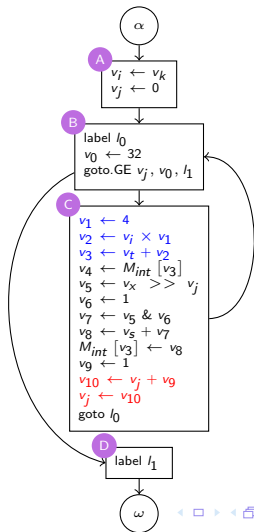
Extraction d'invariant





Optimisation

extraction d'invariant



$$IN_C = \{v_i \leftarrow v_k, v_j \leftarrow v_k, \dots\}$$

$v_1 \leftarrow 4$ invariant (par définition)

$$IN_C = IN_C \cup \{v_1 \leftarrow 4\}$$

$v_2 \leftarrow v_i \times v_1$ invariant car $v_1 \leftarrow 4$ et $v_i \leftarrow 0$ sont hors boucle !

$v_{10} \leftarrow v_j + v_9$ non-invariant car v_j défini par :

- $v_j \leftarrow 0$ hors boucle
- $v_j \leftarrow v_{10}$ dans la boucle

Chaînes utilisation-définition

Propriété recherchée : $v_i \leftarrow v_j \omega v_k$ dans B est invariant $\iff$ toutes les définitions de v_j et v_k dans IN_B sont hors de la boucle.

Quels quadruplets définissent un registre ?

- $\mathcal{D} \subset Q$ – quadruplets du programme
- définition des registres sur un chemin : $\bigcup$
- $KILL_B = \{v \leftarrow e \in \mathcal{D} \mid \exists v \leftarrow e' \in B\}$
- $GEN_B = \{v \leftarrow e \mid v \leftarrow e \in B\}$
- $IN_B = \bigcup_{A \in PRED(B)} OUT_A$
- $OUT_B = (IN_B \setminus KILL_B) \cup GEN_B$
- treillis $(\wp(\mathcal{D}), \subseteq, \cup, \cap, \mathcal{D}, \emptyset)$

Calcul de domination

Propriété : $\forall A, B \in V, A \text{ dom } B \iff \forall \pi \in \alpha \xrightarrow{*} B \implies A \in \pi$
 $\alpha \xrightarrow{*} B$: ensemble des chemins de α à B , V : BB du CFG

IN_B = ensemble des dominants de B

- $A \text{ dom } B \iff A \in IN_B$
- $\mathcal{D} \subset V$
- $KILL_B = \emptyset$
- $GEN_B = \{B\}$ – B peut dominer ses successeurs
- $IN_B = \bigcap_{A \in PRED(B)} OUT_A$ – dominant sur tous les chemins depuis α
- treillis $(\wp(V), \supseteq, \cap, \cup, V, \emptyset)$
- $KILL_\alpha = V, GEN_\alpha = \emptyset$

À quoi ça sert ?

Identification des *têtes de boucle* : $B \in V$ est une tête de boucle
 $\iff \exists A \rightarrow B \in E \wedge B \text{ dom } A$.

$a \rightarrow b$ est alors appelé *arc retour*.



Exercice

Soit le CFG ci-dessous :

• $V = \{A, B, C, D, E, F, G\}$

• $E = \{A \rightarrow D, A \rightarrow G, B \rightarrow A, B \rightarrow E, D \rightarrow B, D \rightarrow C, E \rightarrow D, F \rightarrow B, G \rightarrow A\}$

- 1 Calculer les OUT_B de domination.
- 2 En utilisant les OUT_B , identifiez les têtes de boucle ?

Solution (algo de base)

$PRED_A = \{B, G\}$, $PRED_B = \{F\}$, $PRED_C = \{D\}$, $PRED_D = \{A, E\}$, $PRED_E = \{B\}$, $PRED_F = \emptyset$,
 $PRED_G = \{A\}$,
 $GEN_i = \{i\}$, $KILL_\alpha = V$, $KILL_i = \emptyset$

	l_0 IN_i, OUT_i	l_1 IN_i, OUT_i	l_2 IN_i, OUT_i	l_3 IN_i, OUT_i
A	V, V	V, V	V, V	V, V
B	V, V	V, V	$\{F\}, \{B, F\}$	$\{F\}, \{B, F\}$
C	V, V	V, V	V, V	V, V
D	V, V	V, V	V, V	$\{B, E, F\}, \{B, D, E, F\}$
E	V, V	V, V	$\{B, F\}, \{B, E, F\}$	$\{B, F\}, \{B, E, F\}$
F	$\emptyset, V$	$\emptyset, \{F\}$	$\emptyset, \{F\}$	$\emptyset, \{F\}$
G	V, V	V, V	V, V	V, V

	l_4 IN_i, OUT_i	l_5 IN_i, OUT_i
A	$\{B, F\}, \{A, B, F\}$	$\{B, F\}, \{A, B, F\}$
B	$\{F\}, \{B, F\}$	$\{F\}, \{B, F\}$
C	$\{B, D, E, F\}, \{B, C, D, E, F\}$	$\{B, D, F\}, \{B, C, D, F\}$
D	$\{B, F\}, \{B, D, F\}$	$\{B, F\}, \{B, D, F\}$
E	$\{B, F\}, \{B, E, F\}$	$\{B, F\}, \{B, E, F\}$
F	$\{F\}$	$\emptyset, \{F\}$
G	$\{A, B, F\}, \{A, B, F, G\}$	$\{A, B, F\}, \{A, B, F, G\}$

Solution (algo avec liste de travail)

$PRED_A = \{B, G\}$, $PRED_B = \{F\}$, $PRED_C = \{D\}$, $PRED_D = \{A, E\}$, $PRED_E = \{B\}$,
 $PRED_F = \emptyset$, $PRED_G = \{A\}$
 $GEN_i = \{i\}$, $KILL_\alpha = V$, $KILL_i = \emptyset$

$wl = \{F\} \Rightarrow F : IN_F = V$, $OUT_F = \{F\}$

$wl = \{B\} \Rightarrow B : IN_B = \{F\}$, $OUT_B = \{B, F\}$

$wl = \{A, E\} \Rightarrow A : IN_A = \{B, F\}$, $OUT_A = \{A, B, F\}$

$wl = \{E, D, G\} \Rightarrow E : IN_E = \{B, F\}$, $OUT_E = \{B, E, F\}$

$wl = \{D, G\} \Rightarrow D : IN_D = \{B, F\}$, $OUT_D = \{B, D, F\}$

$wl = \{G, B, C\} \Rightarrow G : IN_G = \{A, B, F\}$, $OUT_G = \{A, B, F, G\}$

$wl = \{B, C, A\} \Rightarrow B : IN_B = \{F\}$ – stop

$wl = \{C, A\} \Rightarrow C : IN_C = \{B, D, F\}$, $OUT_C = \{C, B, D, F\}$

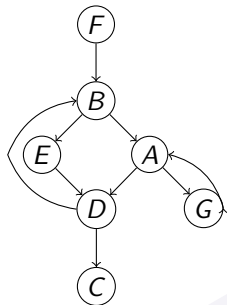
$wl = \{A\} \Rightarrow A : IN_A = \{B, F\}$ – stop

$wl = \emptyset$ – fin.

Solution (têtes de boucle)

$OUT_A = \{A, B, F\}$, $OUT_B = \{B, F\}$, $OUT_C = \{B, C, D, F\}$, $OUT_D = \{B, D, F\}$
 $OUT_E = \{B, E, F\}$, $OUT_F = \{F\}$, $OUT_G = \{A, B, F, G\}$

arc	dom	tête de boucle
$A \rightarrow D$	faux	B
$A \rightarrow G$	faux	
$B \rightarrow A$	faux	
$B \rightarrow E$	faux	
$D \rightarrow B$	vrai	
$E \rightarrow D$	faux	
$F \rightarrow B$	faux	A
$G \rightarrow A$	vrai	



Calcul de vivacité

Propriété : v dans B est vivante s'il y a une utilisation de v dans les chemins partant de de B

$$\forall B \in V, \text{alive}(v, B) \iff v \in IN_B$$

Astuce : parcourir G en arrière – on inverse le CFG.

Domaine de vivacité :

- $\mathcal{D} = \mathcal{V}$ – registres virtuels du programme
- $KILL_B = \{v \in \mathcal{V} \mid v \leftarrow e \in B\}$
- $GEN_B = \{v \in \mathcal{V} \mid v' \leftarrow f(v) \in B\}$
- $IN_B = \bigcup_{A \in \text{SUCC}(B)} OUT_A$ – v vivante sur au moins un des chemins
- treillis $(\wp(\mathcal{V}), \subseteq, \cup, \cap, \mathcal{V}, \emptyset)$



Construction de treillis

treillis connus

- $\mathbb{B}$ – ensemble des booléens
 $\langle \mathbb{B}, \leq, \wedge, \vee, \text{true}, \text{false} \rangle$
 $\langle \mathbb{B}, \geq, \vee, \wedge, \text{false}, \text{true} \rangle$
- ensemble des parties de $\mathcal{X}$
 $\langle \wp(\mathcal{X}), \subset, \cup, \cap, \mathcal{X}, \emptyset \rangle$
- entiers naturels augmentés – $\mathbb{N}^\infty = \mathbb{N} \cup \{+\infty\}$
 $\langle \mathbb{N}^\infty, \leq, \max, \min, \infty, 0 \rangle$
- entiers signés augmentés – $\mathbb{Z}^\infty = \mathbb{Z} \cup \{-\infty, +\infty\}$
 $\langle \mathbb{Z}^\infty, \leq, \max, \min, +\infty, -\infty \rangle$



Construction de treillis

ordre plat

Soit le domaine $\mathcal{S} = \{x_1, x_2, \dots, x_n\}$,
il est toujours possible de construire un
ordre plat :

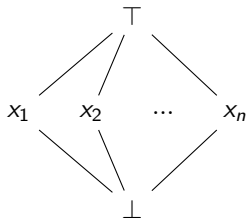
$$\mathcal{D} = \mathcal{S} \cup \{\perp, \top\}$$

Et un treillis :

$$\langle \mathcal{S} \cup \{\perp, \top\}, \sqsubseteq, \sqcup, \sqcap, \perp, \top \rangle$$

Avec $x, y \in \mathcal{D}$:

- $\perp \sqsubseteq x \sqsubseteq \top$
- $x \sqcup y = x$ si $x = y$, $\top$ sinon.
- $x \sqcap y = x$ si $x = y$, $\perp$ sinon.



Application :

- $\mathcal{S} = \mathbb{B} = \{false, true\}$
- $\mathcal{D} = \{\perp, false, true, \top\}$



Construction de treillis

composition

Soit 2 treillis :

- $\langle \mathcal{S}_A, \sqsubseteq_A, \sqcup_A, \sqcap_A, \top_A, \perp_A \rangle$
- $\langle \mathcal{S}_B, \sqsubseteq_B, \sqcup_B, \sqcap_B, \top_B, \perp_B \rangle$

On peut **composer** les 2 treillis :

$$\mathcal{D} = \mathcal{S}_A \times \mathcal{S}_B$$

Et obtenir un nouveau treillis :

$$\langle \mathcal{D}, \sqsubseteq, \sqcup, \sqcap, (\top_A, \top_B), (\perp_A, \perp_B) \rangle$$

Tel que $\forall (x_A, x_B), (y_A, y_B) \in \mathcal{D}$:

- $(x_A, x_B) \sqsubseteq (y_A, y_B) \iff (x_A \neq y_A \wedge x_A \sqsubseteq_A y_A) \vee (x_A = y_A \wedge x_B \sqsubseteq_B y_B)$
- $(x_A, x_B) \sqcup (y_A, y_B) = (x_A \sqcup_A y_A, x_B \sqcup_B y_B)$
- $(x_A, x_B) \sqcap (y_A, y_B) = (x_A \sqcap_A y_A, x_B \sqcap_B y_B)$

Application : paire, triplet, tuple, vecteur



Construction de treillis

fonction

Soit :

- un treillis $\langle \mathcal{S}, \sqsubseteq, \sqcup, \sqcap, \top, \perp \rangle$
- un ensemble $\mathcal{X}$
- une famille de fonctions $\mathcal{F}$ avec $f \in \mathcal{F} : \mathcal{X} \rightarrow \mathcal{S}$

Alors on obtient un nouveau treillis :

$$\langle \mathcal{F}, \sqsubseteq^*, \sqcup^*, \sqcap^*, \lambda x. \top, \lambda x. \perp \rangle$$

Tel que $\forall f_1, f_2 \in \mathcal{F}$:

- $f_1 \sqsubseteq f_2 \iff \forall x \in \mathcal{X}, f_1 \sqsubseteq^* f_2$
- $f_1 \sqcup^* f_2 = \lambda x. f_1(x) \sqcup f_2(x)$
- $f_1 \sqcap^* f_2 = \lambda x. f_1(x) \sqcap f_2(x)$

Application : dictionnaire

Bilan : optimisation

Optimisation :

- = transformation de programme
- qui conserve la sémantique du programme
- qui vérifie certaines conditions
- condition = propriétés obtenues par des analyses du programme

Analyse du programme (analyse statique) :

- propriété = { ensemble de valeurs possibles }
- analyser tous les chemins en parallèle
- usage d'un treillis $\implies$ méthode de calcul et preuve de terminaison

Bilan : Analyse Itérative de Flot de Donnée

- définition du domaine $\mathcal{D}$
- définition de $KILL_b$ et GEN_b
 $OUT_b = (IN_b \setminus KILL_b) \cup GEN_b$
- sens de parcours du CFG
 - en avant : $IN_b = \sqcup_{b' \in PRED(b)} OUT_{b'}$
 - en arrière : $IN_b = \sqcup_{b' \in SUCC(b)} OUT_{b'}$
- agrégation au niveau des chemins
 - sur au moins un des chemins (analyse *MAY*)
 $\sqcup = \cup \implies (\mathcal{D}, \subseteq, \cup, \cap, \emptyset, \mathcal{D})$
 - sur tous les chemins (analyse *MUST*)
 $\sqcup = \cap \implies (\mathcal{D}, \supseteq, \cap, \cup, \mathcal{D}, \emptyset)$



Implantation

$\mathcal{D}$: ensemble de quadruplet, bloc de base, etc

$\nu : \mathcal{D} \rightarrow [0, |\mathcal{D}| - 1]$ – affecte un numéro à chaque $x \in \mathcal{D}$

Formel	Vecteur de bit	En C
$\wp(\mathcal{D})$	$\mathbb{B}^{ \mathcal{D} }$	typedef uint64_t S[N];
$\emptyset$	$[0, 0, \dots, 0]$	$S_s = \{0, 0, \dots, 0\}$
$\{x\}$	$\emptyset [\nu(x) \mapsto 1]$	$s[i \gg 6] = 1 \ll (i \& 0x3f);$
$S \cup S'$	$[S[i] \vee S'[i]]_{0 \leq i < \mathcal{D} }$	$s[i] sp[i]$
$S \cap S'$	$[S[i] \wedge S'[i]]_{0 \leq i < \mathcal{D} }$	$s[i] \& sp[i]$
$S \setminus S'$	$[S[i] \wedge \neg S'[i]]_{0 \leq i < \mathcal{D} }$	$s[i] \& \sim sp[i]$
$x \in S$	$s[\nu(x)]$	$(s[i \gg 6] \& (1 \ll (i \& 0x2f))) != 0$



Bilan : on sait tout faire ?

- Non !
- problème : ensemble infinis $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{R}$, ...

```
int sum(int n) {  
    int s = 0;  
    while(n > 0) {  
        s += n;  
        n--;  
    }  
    return s;  
}
```

Valeurs de s ou n ?

- ensembles abstraits $\Rightarrow$ *Interprétation Abstraite* (Cousot, 1978)



Outline

- ① Optimisations locales
- ② Optimisations globales
- ③ Interprétation abstraite**
- ④ Conclusion



Exemple

```
int f(unsigned int x, int y) {  
    if (y > 0) {  
        if (x * y + 1 < 0)  
            while (x < 0)  
                x = x + sqrt(x);  
        return x;  
    }  
    return y;  
}
```

Question : y a-t-il du code mort dans ce code ?

- $y > 0$ toujours vrai ou faux ?
- $x * y + 1 < 0$ toujours vrai ou faux ?
- $x < 0$ toujours faux ?

Attention : toujours vrai/faux $\neq$ parfois vrai, parfois faux.

Faisons l'analyse...

Proposition 1 : examinons toutes les valeurs des variables.

$$\mathcal{D} = ID \rightarrow \mathbb{Z}$$

Problème : x et y multiples valeurs.

Proposition 2 : testons toutes les valeurs possibles de x et y .

Problème : même sur 32-bit $\Rightarrow 2^{32} \times 2^{32} = 2^{64}$ configurations à tester !

Proposition 3 : considérons des ensembles bien choisis de valeurs et voyons si nous pouvons calculer avec.

$$\mathcal{D}_{sign} = \{\perp, n, 0, p, \top\}$$

avec $p \mapsto \mathbb{Z}^+$, $0 \mapsto \{0\}$, $n \mapsto \mathbb{Z}^-$, $\top \mapsto \mathbb{Z}$ et $\perp \mapsto \emptyset$.



Peut-on calculer avec $\mathcal{D}_{sign}$?

$\boxplus$	$\perp$	n	0	p	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
n	$\perp$	n	n	$\top$	$\top$
0	$\perp$	n	0	p	$\top$
p	$\perp$	$\top$	p	p	$\top$
$\top$	$\perp$	$\top$	$\top$	$\top$	$\top$

$\boxtimes$	$\perp$	n	0	p	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
n	$\perp$	p	0	n	$\top$
0	$\perp$	0	0	0	0
p	$\perp$	n	0	p	$\top$
$\top$	$\perp$	$\top$	0	$\top$	$\top$

Appliqué à notre analyse

```
int f(unsigned int x, int y) {  
    // {x ↦ p, y ↦ ⊤}  
    if (y > 0) {  
        // {x ↦ p, y ↦ p}  
        if (x * y + 1 < 0)  
            // {x ↦ p, y ↦ p}  
            while (x < 0)  
                // {x ↦ ⊥, y ↦ p}  
                x = x + sqrt(-x);  
        return x;  
    }  
    return y;  
}
```

State $\mathcal{S}_{sign} : ID \rightarrow \mathcal{D}_{sign}$

$$y > 0 = \top > 0$$

⇒ on ne peut rien conclure.

$$\begin{aligned}x * y + 1 < 0 &\iff p \boxtimes p \boxplus p < 0 \\&\iff p \boxplus p < 0 \\&\iff p < 0 \\&\iff \text{false}\end{aligned}$$

⇒ code mort !

{x ↦ ⊥, y ↦ p} – état indéfini :
boucle infaisable (morte).

Autre exemple

```
// {x ↦ T}
if (x < 0)
  // {x ↦ n}
  x = -x;
  // s1 = {x ↦ p}
else
  // s2 = {x ↦ T}
  ;
// s = s1 ⊔ s2
// s = {x ↦ p ⊔ T}
// s = {x ↦ T}
```

$\sqcup$	$\perp$	n	0	p	T
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
n	$\perp$	n	T	T	T
0	$\perp$	T	0	T	T
p	$\perp$	T	T	p	T
T	$\perp$	T	T	T	T

Beaucoup de T !

Amélioration :

$\mathcal{D}_{sign}^* = \{\perp, p, p^*, 0, p^*, p\}$

avec $p^* = \mathbb{Z}^+ \cup \{0\}$ et $n^* = \mathbb{Z}^- \cup \{0\}$.

Ordre partiel, treillis

$\sqcup$ définit un ordre partiel :

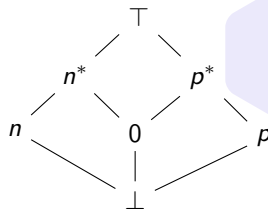
$$\forall a, b \in \mathcal{D}_{sign}^*, a \sqsubseteq b \iff a \sqcup b = b$$

On peut même définir l'opération inverse :

$$\forall a, b \in \mathcal{D}_{sign}^*, a \sqcap b = a \iff a \sqsubseteq b$$

Et obtient un treillis :

$$\langle \mathcal{D}_{sign}^*, \sqcup, \sqcap, \sqsubseteq, \top, \perp \rangle$$



Autre exemple (corrigé)

```
// {x ↦ T}
if (x < 0)
// {x ↦ n}
x = -x;
// s1 = {x ↦ p}
else
// s2 = {x ↦ p*}
;
// s = s1 ⊔ s2
// s = {x ↦ p ⊔ p*}
// s = {x ↦ p*() }
```

$\sqcup$	$\perp$	n	n^*	0	p^*	p	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
n	$\perp$	n	n^*	n^*	$\top$	$\top$	$\top$
n^*	$\perp$	n	n^*	n^*	$\top$	$\top$	$\top$
0	$\perp$	n^*	n^*	0	p^*	p^*	$\top$
p^*	$\perp$	$\top$	$\top$	p^*	p^*	p^*	$\top$
p	$\perp$	$\top$	$\top$	p^*	p^*	p	$\top$
$\top$	$\perp$	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$

Beaucoup de $\top$!

Amélioration :

$\mathcal{D}_{sign}^* = \{\perp, p, p^*, 0, p^*, p\}$

avec $p^* = \mathbb{Z}^+ \cup \{0\}$ et $n^* = \mathbb{Z}^- \cup \{0\}$.

Remarque : la précision de l'analyse dépend du domaine.

Abstraire, concrétiser

Pour un programme P .

Interpréter une constante :

$$\alpha : \mathbb{Z} \rightarrow \mathcal{D}_{sign}^* \text{ (abstraction)}$$

$$\alpha(0) = 0$$

$$\alpha(+3) = p$$

$$\alpha(-5) = n$$

Sémantique concrète :

$$\llbracket P \rrbracket_\gamma : \mathcal{S} \rightarrow \mathcal{S}$$

$$\text{avec } \mathcal{S} = ID \rightarrow \mathbb{Z}$$

$$\text{Exemple : } \llbracket x + y \rrbracket_\gamma \Leftrightarrow \llbracket x \boxplus y \rrbracket_\alpha$$

$$\text{Sémantique collective : } \mathcal{S}_\# = \wp(ID \rightarrow \mathbb{Z}), \llbracket P \rrbracket_\# : \mathcal{S}_\# \rightarrow \mathcal{S}_\#$$

Revenir dans le domaine concret :

$$\gamma : \mathcal{D}_{sign}^* \rightarrow \wp(\mathbb{Z}) \text{ (concrétisation)}$$

$$\gamma(0) = \{0\}$$

$$\gamma(p) = \mathbb{Z}^+$$

$$\gamma(n) = \mathbb{Z}^-$$

$$\gamma(\top) = \mathbb{Z}$$

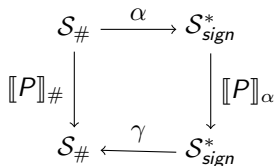
Sémantique abstraite :

$$\llbracket P \rrbracket_\alpha : \mathcal{S}_{sign}^* \rightarrow \mathcal{S}_{sign}^*$$

$$\text{avec } \mathcal{S}_{sign}^* = ID \rightarrow \mathcal{D}_{sign}^*$$



Pour résumer...



Est-ce que $\llbracket P \rrbracket_{\alpha}$ est valide ?

Oui si le résultat abstrait contient tous les résultats concrets (je n'en ai pas oublié!).

$$\forall s \in S_{\#}, \llbracket P \rrbracket_{\#}(s) \subseteq \gamma(\llbracket P \rrbracket_{\alpha}(\alpha(s)))$$

connection de Galois : cela est vrai si α , $\llbracket P \rrbracket_{\alpha}$, γ sont monotones.

$F : \mathcal{D}_1 \rightarrow \mathcal{D}_2$ est monotone $\iff \forall x, y \in \mathcal{D}_1, x \sqsubseteq y \implies F(x) \subseteq F(y)$.

Pour les boucles...

Corollaire : $\sqcup$, $\boxplus$, $\boxtimes$, ... doivent être monotones sur $\mathcal{D}_{sign}^*$.

Soit le programme P :

```
while ( i < 10 )  
    i ++;
```

Soit f l'interprétation de la boucle : $f(s) = \llbracket i < 10 ; i++ \rrbracket_{\alpha}(s)$

L'état en début de boucle (avant condition) est :

$$\begin{aligned}\forall s \in \mathcal{S}_{sign}^*, \llbracket P \rrbracket_{\alpha} &= s \sqcup f(s) \sqcup f(f(s)) \sqcup f(f(f(s))) \sqcup \dots \\ &= \bigsqcup_{i \in \mathbb{N}} f^i(s) \\ &= \llbracket f \rrbracket_{fixpoint}(s)\end{aligned}$$

f monotones sur $\mathcal{S}_{sign}^*$ qui a $\top$: cela termine toujours !



Exercice

Faire l'interprétation sur $\mathcal{S}_{sign}^*$ du programme ci-dessous et détectez les conditions qui sont toujours vraies ou toujours fausses.

```
int f(int x, int n) {  
    int s = 0;  
    int i = 0;  
    while(i < 10) {  
        s = s + x * x;  
        if(s & 1 == 1)  
            s = s + 1;  
    }  
    if(s >= 0)  
        return 1;  
    else  
        return -1;  
}
```

Autre abstraction : les intervalles

Définitions

$$\mathcal{D}_{int} = \{[a, b] \mid a \in \mathbb{Z}^{-\infty}, b \in \mathbb{Z}^{+\infty}\}$$

$$\text{avec } \mathbb{Z}^{-\infty} = \mathbb{Z} \cup \{-\infty\}$$

$$\text{et } \mathbb{Z}^{+\infty} = \mathbb{Z} \cup \{+\infty\}$$

$$\top = [-\infty, +\infty], \perp = [+ \infty, -\infty]$$

$$\forall n \in \mathbb{Z}, \alpha(n) = [n, n]$$

$$\forall [a, b] \in \mathcal{D}_{int},$$

$$\gamma([a, b]) = \{x \in \mathbb{Z} \mid a \leq x \leq b\}$$

$$[a, b] \sqsubseteq [a', b'] \iff a' \leq a \wedge b \leq b'$$

$$[a, b] \sqcup [a', b'] = [\min(a, a'), \max(b, b')]$$

$$[a, b] \sqcap [a', b'] = [\max(a, a'), \min(b, b')]$$

$\Rightarrow \langle \mathcal{D}_{int}, \sqsubseteq, \sqcup, \sqcap, \top, \perp \rangle$ est un treillis !

Fonctions d'interprétation

$$\forall [a, b], [a', b'] \in \mathcal{D}_{int},$$

$$\bullet [a, b] \boxplus [a', b'] = [a + a', b + b']$$

$$\bullet [a, b] \boxtimes [a', b'] = [\min(aa', ab', ba', bb'), \max(aa', ab', ba', bb')]$$

Exemple

```
int f(int *t, int n) {  
    int s = 0, i = 0;  
    /* (a) */  
    while(i < n) {  
        /* (b) */  
        s += t[i];  
        i++;  
        /* (c) */  
    }  
}
```

```
int t[10];  
f(t, 11);
```

Calcul

$$s_f = \{n \mapsto [11, 11]\}$$

$$s_a = \llbracket s = 0; i = 0 \rrbracket_{int}(s_{in})$$

$$s_b = \llbracket i < n \rrbracket_{int}(s_a \sqcup s_c)$$

$$s_c = \llbracket s += t[i]; i ++ \rrbracket_{int}(s_b)$$

Résultat de l'analyse

$$(a) \{n \mapsto [11, 11], i \mapsto [0, 0]\}$$

$$(b) \{n \mapsto [11, 11], i \mapsto [0, 10]\}$$

$$(c) \{n \mapsto [11, 11], i \mapsto [1, 11]\}$$

Note : s non analysé.

⇒ erreur mémoire sur t[i] (b).

Exemple détaillé

```
int f(int *t, int n) {  
    int s = 0, i = 0;  
    /* (a) */  
    while(i < n) {  
        /* (b) */  
        s += t[i];  
        i++;  
        /* (c) */  
    }  
}  
  
int t[10];  
f(t, 11);
```

Résultat de l'analyse

(a) $\{n \mapsto [11, 11], i \mapsto [0, 0]\}$

(b) $\{n \mapsto [11, 11], i \mapsto [0, 0]\}$

(c) $\{n \mapsto [11, 11], i \mapsto [1, 1]\}$

(b) $\{n \mapsto [11, 11], i \mapsto [0, 1]\}$

(c) $\{n \mapsto [11, 11], i \mapsto [1, 2]\}$

(b) $\{n \mapsto [11, 11], i \mapsto [0, 2]\}$

(c) $\{n \mapsto [11, 11], i \mapsto [1, 3]\}$

...

(c) $\{n \mapsto [11, 11], i \mapsto [1, 11]\}$

(b) $\{n \mapsto [11, 11], i \mapsto [0, 10]\}$

(c) $\{n \mapsto [11, 11], i \mapsto [1, 11]\}$

$\Rightarrow$ point fixe !

C'est long !



Opération d'élargissement (widening)

Définition

$[a, b] \nabla [a', b'] = [a'', b'']$ tel que

$$a'' = \begin{cases} -\infty & \text{si } a' < a \\ \min(a, a'') & \text{sinon} \end{cases}$$

$$b'' = \begin{cases} +\infty & \text{si } b' > b \\ \max(b, b') & \text{sinon} \end{cases}$$

Propriétés : $[a, b] \sqcup [a', b'] \sqsubseteq [a, b] \nabla [a', b']$

Utilisation : dans les boucles !

Exemple avec élargissement

```
int f(int *t, int n) {  
    int s = 0, i = 0;  
    /* (a) */  
    while(i < n) {  
        /* (b) */  
        s += t[i];  
        i++;  
        /* (c) */  
    }  
}
```

```
int t[10];  
f(t, 11);
```

Résultat de l'analyse Calcul

$$s_0 = \{n \mapsto \perp, i \mapsto \perp\}$$

$$s_{in} = \{n \mapsto [11, 11]\}$$

$$s_a = \llbracket s = 0; i = 0 \rrbracket_{int}(s_{in})$$

$$s_i = s_{i-1} \nabla (s_a \sqcup s_c)$$

$$s_b = \llbracket i < n \rrbracket_{int}(s_i)$$

$$s_c = \llbracket s += t[i]; i ++ \rrbracket_{int}(s_b)$$

Résultat de l'analyse

$$s_a = \{n \mapsto [11, 11], i \mapsto [0, 0]\}$$

$$s_1 = \{n \mapsto [11, 11], i \mapsto [0, 0]\}$$

$$s_b = \{n \mapsto [11, 11], i \mapsto [0, 0]\}$$

$$s_c = \{n \mapsto [11, 11], i \mapsto [1, 1]\}$$

$$s_2 = \{n \mapsto [11, 11], i \mapsto [0, +\infty]\}$$

$$s_b = \{n \mapsto [11, 11], i \mapsto [0, 10]\}$$

$$s_c = \{n \mapsto [11, 11], i \mapsto [1, 11]\}$$

$$s_3 = \{n \mapsto [11, 11], i \mapsto [0, +\infty]\}$$



Exercice 1

```
int f(int x) {  
    int y = x;  
    if(x < 0)  
        y = -x;  
    return y;  
}
```

En utilisant l'analyse par intervalle, montrer que y est toujours positif à la fin de la fonction.



Exercice 2

```
label f
 $r_t \leftarrow \&tab$ 
 $r_i \leftarrow 0$ 
 $r_s \leftarrow 0$ 
label  $L_0$ 
 $r_0 \leftarrow 20$ 
goto.ge  $r_i, r_0, L_1$ 
```

```
 $r_1 \leftarrow 4$ 
 $r_2 \leftarrow r_i \times r_1$ 
 $r_3 \leftarrow r_t + r_2$ 
 $r_4 \leftarrow M_{int}[r_3]$ 
 $r_s \leftarrow r_s = r_4$ 
 $r_5 \leftarrow 1$ 
 $r_i \leftarrow r_i + r_5$ 
goto  $L_0$ 
label  $L_1$ 
```

Calculer le taux de hit de l'accès mémoire dans le cache de donnée (en considérant des blocs de taille 16o).

Utilisation pratique

Transformations de boucle (grands tableaux) :

- fusion de boucle
- *tiling*
- restructuration

Analyse complexes :

- dépendances inter-itérations

```
for(i = 0; i < N; i++)  
  t[i] = a[i] + b[i];  
for(j = 0; j < N; j++)  
  t[i] = t[i] + 1;
```

```
for(i = 0; i < N; i++) {  
  t[i] = a[i] + b[i];  
  t[i] = t[i] + 1;  
}
```



Outline

- ① Optimisations locales
- ② Optimisations globales
- ③ Interprétation abstraite
- ④ Conclusion**



Conclusion